A Minigrid-Graph Sudoku Solver

Holistic Design, Exact Support Pruning, and Analysis Workflow

Abstract

This paper describes the full `sudoku_solver` project as an end-to-end Sudoku research and engineering system. The solver decomposes a 9×9 board into minigrids, generates valid box-level permutations, constructs a compatibility graph across dependent minigrids, prunes unsupported configurations, and extracts complete solutions. In addition to the core algorithm, the project includes dataset analysis tooling, reproducible figure generation, and a paper pipeline intended to evolve as new measurements and scripts are added. The document unifies the previously isolated branch-free relationship note into a broader project view.

1 Project Overview

The repository is organized around five solver phases and a paper-analysis pipeline:

1. Mask initialization
2. Minigrid permutation generation
3. Compatibility graph construction
4. Exact support pruning
5. Solution extraction

Supporting infrastructure includes sample datasets, CSV analysis outputs, plotting scripts, and LaTeX sources.

2 Motivation

Traditional Sudoku solvers often rely on direct backtracking over cells. This project instead treats each 3×3 minigrid as the primary combinational unit. That shift makes it possible to:

- exploit box-local permutation structure,
- precompute row and column compatibility masks,
- reason about global consistency through a graph of minigrid permutations,
- analyze solver behavior phase-by-phase across puzzle datasets.

3 Architecture

3.1 Phase 1: Mask Initialization

The board is parsed into row, column, and box conflict masks. These masks provide the legality checks used during local permutation generation.

3.2 Phase 2: Minigrid Permutation Generation

For each minigrid, the solver runs a depth-first search with a minimum-remaining-values heuristic. The output of this phase is a set of valid candidate fillings for each minigrid. These candidates are represented as permutation nodes containing both cell assignments and row/column masks.

3.3 Phase 3: Graph Construction

Each permutation node becomes a graph vertex. Edges connect vertices in dependent minigrids when row or column masks are compatible. The project retains a specialized branch-free relationship primitive, documented separately in `minigrid_relationship.tex`, for identifying whether two minigrids share row or column constraints.

3.4 Phase 4: Exact Support Pruning

Earlier connectivity-only pruning was too aggressive on realistic puzzles because local degree conditions do not imply membership in a globally consistent full-board assignment. The current implementation instead performs exact support pruning: it explores compatible minigrid configurations, marks permutations that appear in at least one full assignment, and discards unsupported nodes. This makes pruning semantically aligned with actual solvability.

3.5 Phase 5: Solution Extraction

After pruning, the remaining graph is searched for complete assignments. Valid assignments are reconstructed into Sudoku boards and classified as unsolvable, unique, or ambiguous.

4 Analysis Pipeline

The project includes a structured analysis path:

- `examples/analyze_dataset.rs` generates CSV outputs,
- `results/` stores analysis outputs,
- `scripts/generate_figures.py` produces paper figures,
- `scripts/run_paper_pipeline.sh` ties analysis, plotting, and paper generation together.

Current analysis outputs include:

- per-difficulty CSV files,
- `complete_analysis.csv`,
- `phase_breakdown.csv`,
- `statistics_summary.txt`.

5 Generated Figures

6 Current Status

The implementation now includes all five phases and integration tests for representative easy and medium sample puzzles. The paper pipeline is intended to remain open-ended: additional scripts, more detailed analysis, and further figures can be added without restructuring the repository again.

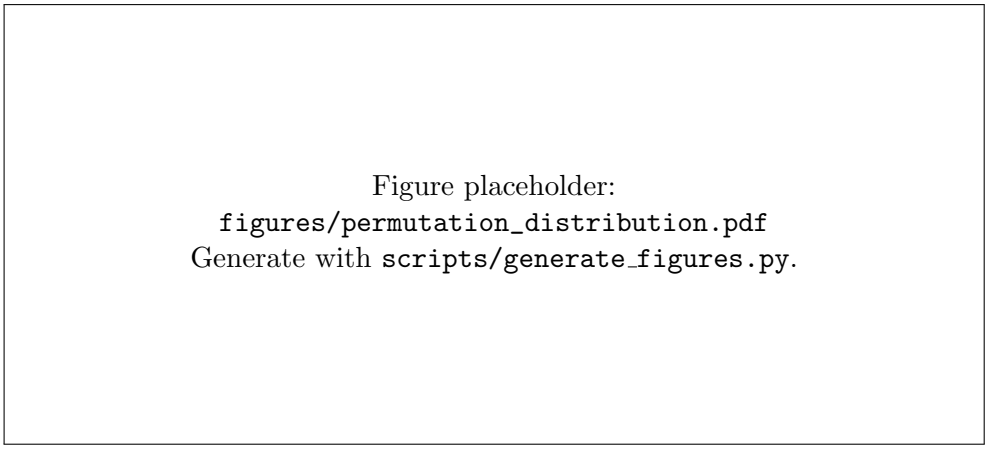

Figure 1: Distribution of minigrid permutation counts.

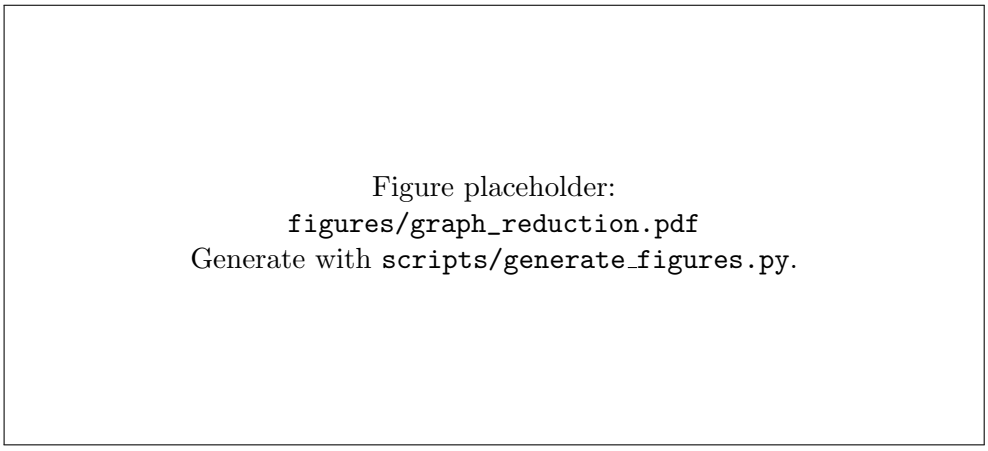

Figure 2: Average graph size before and after pruning.

7 Future Work

Near-term work includes:

- larger dataset runs,
- optimization of exact support pruning for harder puzzles,
- richer benchmark suites,
- expanded plots and tables for publication.

Longer-term work may include new data analysis scripts, solver variants, or comparative baselines against conventional cell-level backtracking approaches.

8 Conclusion

The repository has evolved from a narrowly focused algorithm note into a complete solver-and-analysis project. The full paper source introduced here is meant to become the main narrative document for that broader effort, while the original minigrid relationship note remains a focused reference on one important low-level optimization.

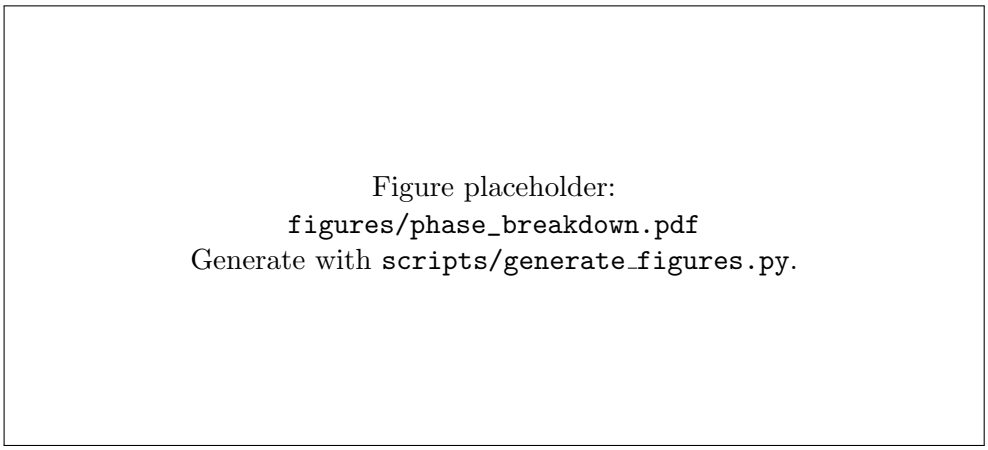

Figure 3: Average time spent in each phase by difficulty.

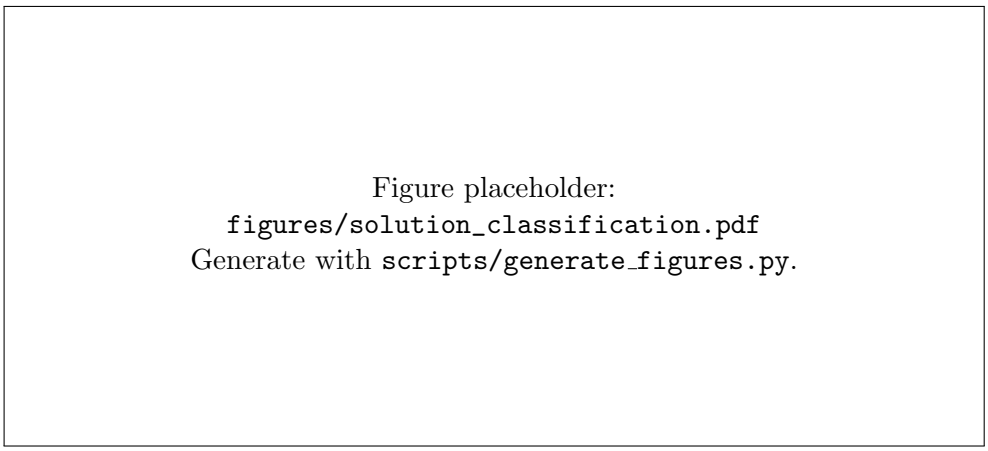

Figure 4: Solution classification across analyzed puzzles.